Fix Ticker Scrolling

ChatGPT - September 8, 2026 at 06:04 PM

CHATGPT

Yes — **that result is actually good news.** It proves the plugin is detecting your selection and changing the state to `paused`.

So the problem is no longer:

> "Why isn't the plugin detecting selection?"

It is detecting it.

The real problem is:

> **What is causing the selection to disappear while VTE is autoscrolling, even though our state remains paused?**

And that also explains why disabling `TMOUT/ALRM` completely made the selection stable: the Zsh ticker is still capable of interfering with VTE's native selection/autoscroll lifecycle.

Why root appeared different

It probably isn't actually a root-vs-REMAC `.zshrc` difference.

Both shells have:

```
Zsh

  TMOUT=1
  trap '_zsh_live_clock_ticker' ALRM
```

Both ultimately execute:

```
Zsh

  zle reset-prompt
```

The plugin writes the state based on the PTY. Your test only proves `/dev/pts/0` behaves correctly. We should test root's actual PTY too, because root will have a different `/dev/pts/N`.

In the **root terminal**, run:

```
Bash
```

```
tty
```

If root says, for example:

```
/dev/pts/1
```

then from another terminal:

Bash

```
watch -n 0.1 'printf "ROOT STATE="; cat /tmp/zsh-live-clock-1 2>/dev/null ||
echo MISSING'
```

Then select in root.

If it says:

```
ROOT STATE=paused
```

while you're holding the mouse, we've confirmed **root and REMAC are behaving identically at the plugin level**.

How I would fix the actual selection problem

I would make one architectural change:

Don't let the Zsh clock depend on `zle reset-prompt` while a mouse operation is occurring.

Right now we have:

```
VTE mouse selection
       ↓
plugin notices selection
       ↓
```

```
/tmp/... = paused
       ↓
Zsh's ALRM fires
       ↓
_zsh_live_clock_ticker()
       ↓
checks state
       ↓
possibly zle reset-prompt
```

Even though the plugin eventually says `paused`, there is still a race between **GTK/VTE's selection/autoscroll processing** and **Zsh's asynchronous ALRM/ZLE processing**.

For long autoscrolling selections, that race is exactly what we don't want.

Instead, I want the plugin to maintain a **hard per-PTY lock**, and the Zsh ticker should check that lock before doing *anything*.

But there's an even cleaner solution:

During an active mouse selection, don't execute ZLE at all.

The ticker should effectively become:

```
Zsh

  if terminal_is_mouse_locked; then
      return
  fi

  zle reset-prompt
```

And the plugin should set that lock **on button press**, not only after `selection-changed`.

That means the sequence becomes:

```
LMB DOWN
   ↓
plugin immediately locks ticker
   ↓
VTE begins selection
   ↓
```

```
VTE autoscrolls
   ↓
Zsh ticker cannot reset prompt
   ↓
LMB UP
   ↓
plugin copies selection
   ↓
plugin clears selection
   ↓
plugin unlocks ticker
   ↓
if at bottom → clock resumes
```

That's much safer than using `get_has_selection()` as the primary protection.

But there's one more thing

Your current plugin does this on release:

```python
GLib.idle_add(
    self._finish_selection,
    vte,
    ident
)
```

and `_finish_selection()` does:

```python
vte.copy_clipboard()
vte.unselect_all()
```

That is probably correct for your desired **copy + immediately remove highlight** behavior.

We should **not remove that**.

The thing I want to change is specifically the **clock lock**, not your selection handling.

So we're at the point where I would modify only two things

Plugin:

- LMB press = hard pause
- mouse wheel = pause
- LMB release = copy → clear highlight → determine bottom → resume
- keep polling as a safety mechanism
- never prematurely unlock while the button is held

Zsh:

Keep your root `.zshrc` exactly as it is, including the root-specific:

```
Zsh

  setopt MENU_COMPLETE
  zstyle ':completion:*' menu false
```

and keep REMAC's completion completely separate.

Don't change either completion engine.

The next step should therefore be to replace the current stable plugin with a **v2 specifically designed around the hard mouse lock**, rather than continuing to alter `.zshrc` .

If you want, I can give you the **complete replacement** `live_clock_scroll.py` , not just a fragment, so we can test this systematically.

YOU

:M- d: -+ - ┌──[Tue Sep 08 3:48:26 PM][root@Intel5]─[/home/remac] └──╼[REMAC]# tty /dev/pts/8 ┌──[Tue Sep 08 3:49:14 PM][root@Intel5]─[/home/remac] └──╼[REMAC]# tty /dev/pts/8 ┌──[Tue Sep 08 3:49:28 PM][root@Intel5]─[/home/remac] └──╼[REMAC]# this still says live even on selecting in te other terminal portion Every 0.1s: printf "ROOT STATE="; cat /tmp/zsh-live-clock-8 2>/dev/null || echo MISSING Intel5: Tue Sep 8 15:49:43 2026 ROOT STATE=live

CHATGPT

Aha. **Now we've found a real difference.**

Your root shell is `/dev/pts/8`, but:

```
ROOT STATE=live
```

even when you're selecting in the *other terminal*.

That is actually expected if you're selecting in the other terminal. **Each terminal has its own PTY and its own state file.**

So if you're selecting in `/dev/pts/0`, you should watch:

```Bash
cat /tmp/zsh-live-clock-0
```

If you're selecting in `/dev/pts/8`, watch:

```Bash
cat /tmp/zsh-live-clock-8
```

The important question is therefore: **does root's own state become `paused` when you select inside the root terminal?**

Do this exact test:

1. Keep the root terminal open

It is:

```
/dev/pts/8
```

2. From another terminal, run

Bash

```
watch -n 0.1 'printf "ROOT STATE="; cat /tmp/zsh-live-clock-8 2>/dev/null ||
echo MISSING'
```

3. Now go physically to the `/dev/pts/8` root terminal

Do a selection **inside that root terminal**.

While holding LMB, check the watcher.

It should change:

```
ROOT STATE=live
```

to:

```
ROOT STATE=paused
```

If it does, **root's plugin integration is working perfectly**.

But there's an important distinction

You said:

> "how should we get it working for scroll highlight selection?"

I think this is the actual issue we're chasing.

There are **two separate things**:

Normal selection

You click and drag over text that's already visible.

The plugin detects it:

```
live → paused
```

and eventually:

```
copy → unselect → live
```

That's working.

Long scroll-selection/autoscroll

You:

1. Hold LMB.
2. Drag toward the bottom/top edge.
3. VTE starts autoscrolling.
4. You keep holding LMB for 10+ seconds.
5. Selection suddenly disappears before release.

That is the broken case.

And your `/dev/pts/0` test already showed that the state becomes `paused` during selection. So the plugin *does* know you're selecting.

That means the next thing we need to investigate is **whether the Zsh process is actually receiving an ALRM and executing `reset-prompt` despite the state being paused.**

I don't want to guess anymore.

We can prove it.

Temporarily change only the ticker in **both root and REMAC** from:

```Zsh
zle reset-prompt
```

to:

```Zsh
print -u2 "CLOCK TICK: $(tty) state=$state"
zle reset-prompt
```

But don't leave that permanently — it's only a diagnostic.

Then do the long selection.

If you see clock-tick messages being emitted while holding LMB, we'll know the ticker is still firing during the selection. If it doesn't emit them, then we know VTE itself is terminating the selection and we need to attack the GTK/VTE event handling instead.

However, there's an even cleaner diagnostic: since you already proved that:

```
Zsh

  TMOUT=0
  trap - ALRM
```

makes long selection stable, we already have very strong evidence that ALRM/ZLE is involved.

So I would now stop changing the root completion configuration entirely.

Your root completion:

```
Zsh

  setopt MENU_COMPLETE
  zstyle ':completion:*' menu false
  zle menu-complete
```

can stay exactly as it is.

Next fix = make the clock ticker completely incapable of touching ZLE during an active mouse drag/autoscroll.

And importantly, we should make that lock happen **on the initial button press**, rather than waiting for `selection-changed`. That's the part I would change in the plugin next.

YOU

just see, it did not show paused

CHATGPT

Yep — I can see exactly what happened in the screenshot.

The `ROOT STATE=live` result is not proving that root selection detection is broken. Your watcher is watching:

```
Bash

  /tmp/zsh-live-clock-8
```

which corresponds to:

```
  /dev/pts/8
```

That's the **bottom-left root terminal**.

But the selection you're testing in the screenshot appears to be happening in the **other terminal pane**, whose PTY is `/dev/pts/0`. Therefore `/tmp/zsh-live-clock-8` correctly remains `live`.

Let's remove all ambiguity

In the **root terminal itself** (bottom-left), run:

```
Bash

  tty
```

You should get:

```
  /dev/pts/8
```

Then in the **bottom-right watcher**, keep:

```
Bash
```

```
watch -n 0.1 'printf "ROOT STATE="; cat /tmp/zsh-live-clock-8 2>/dev/null ||
echo MISSING'
```

Now, **without touching the top terminal**, use the mouse in the bottom-left root terminal:

1. Hold **left mouse button**
2. Select some text
3. Keep holding LMB
4. Watch the bottom-right pane

You should see:

```
ROOT STATE=paused
```

while the selection is active.

But here's the important part

We already know the bigger problem isn't simply "root doesn't detect selection."

You proved earlier that:

```
Zsh

  TMOUT=0
  trap - ALRM
```

makes the **long scrolling selection work correctly**.

That means our real fix should be:

> **The plugin must switch the PTY to `paused` immediately on mouse-button press, rather than waiting for the 100 ms polling cycle.**

Right now your plugin does this:

```
Python
```

```
    def _press(self, vte, event):
        if event.button == 1:
            self.selecting.add(id(vte))

        return False
```

It records `selecting`, **but it doesn't immediately write `paused` to the state file**.

The `_poll()` has to discover it up to 100 ms later.

That's the race we should eliminate.

So don't change either `.zshrc`

Your root completion configuration is fine. Your REMAC completion configuration is fine.

The next plugin version should do this sequence:

```
LMB press
   ↓
IMMEDIATELY write paused
   ↓
VTE performs selection/autoscroll
   ↓
Zsh ticker sees paused
   ↓
NO zle reset-prompt during the drag
   ↓
LMB release
   ↓
copy selection
   ↓
clear highlight
   ↓
wait until release processing is completely finished
   ↓
if at bottom → live
if scrolled → paused
```

And similarly, a mouse-wheel event should immediately force `paused`.

That's the implementation I would change now. The screenshot actually helps confirm that we shouldn't start messing with the root `.zshrc`; we're dealing with the plugin's timing/state transition.